

PyMPDATA, PySDM & Numba-MPI open-source projects

Sylwester Arabas & Kamil Kania

AGH University of Krakow, Poland

Sep 16 2026, MPI-DS, Göttingen



PyMPDATA



PySDM



Numba-MPI

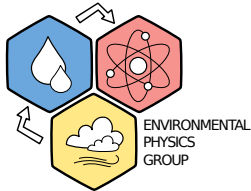


AGH Wissenschaftlich-Technische Universität

(ca. 23 000 students & 2 400 academic/research staff
est. 1913 by emperor Franz Joseph of Austria)



AGH Wissenschaftlich-Technische Universität



Die **AGH Wissenschaftlich-Technische Universität**^[4] (polnisch: *Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie* = Akademie für Bergbau und Hüttenwesen „Stanisław Staszic“ zu Krakau) – kurz **AGH** – ist die größte technische Universität in der Stadt Krakau mit 23.570 Studenten und 2.154 wissenschaftlichen Mitarbeitern.^[2] Sie hat den Ruf einer der besten Universitäten in Polen.^[5]

Die AGH Wissenschaftlich-Technische Universität wurde 1919 gegründet und das Hauptgebäude wurde in den Jahren 1923–1935 in Czarna Wieś errichtet. Rektor der Universität ist seit Mai 2020 Jerzy Lis.^{[6][7]}

Fakultäten

Die AGH Universität gliedert sich in 16 Fakultäten^[7]:

- Fakultät für Bauingenieurwesen und Ressourcenmanagement (*Wydział Inżynierii Łądowej i Gospodarki Zasobami*)
- Fakultät für Metalltechnik und Industriellinformatik (*Wydział Inżynierii Metali i Informatyki Przemysłowej*)
- Fakultät für Elektrotechnik, Automatikk, Informatik und Biomedizinische Technik (*Wydział Elektrotechniki, Automatyki, Informatyki i Inżynierii Biomedycznej*)
- Fakultät für Informatik, Elektronik und Telekommunikation (*Wydział Informatyki, Elektroniki i Telekomunikacji*)
- Fakultät für Maschinenbau und Robotik (*Wydział Inżynierii Mechanicznej i Robotyki*)
- Fakultät für Geologie, Geophysik und Umweltschutz (*Wydział Geologii, Geofizyki i Ochrony Środowiska*)
- Fakultät für montane Geodäsie und Umweltingenieurwesen (*Wydział Geodezji Górniczej i Inżynierii Środowiska*)
- Fakultät für Material- und Keramikwissenschaften (*Wydział Inżynierii Materiałowej i Ceramiki*)
- Fakultät für Gießereitechnik (*Wydział Odlewnictwa*)
- Fakultät für Nichteisenmetalle (*Wydział Metali Nieżelaznych*)
- Fakultät für Bohr-, Öl- und Gaswesen (*Wydział Wiertnictwa, Nafty i Gazu*)
- Fakultät für Management (*Wydział Zarządzania*)
- Fakultät für Energie und Kraftstoffe (*Wydział Energetyki i Paliw*)
- Fakultät für Physik und Angewandte Informatik (*Wydział Fizyki i Informatyki Stosowanej*)
- Fakultät für angewandte Mathematik (*Wydział Matematyki Stosowanej*)
- Fakultät für Geisteswissenschaften (*Wydział Humanistyczny*)

AGH Wissenschaftlich-Technische Universität
Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie



Motto

„Labore creat, labori et scientiae servis“

Gründung

1919

Trägerschaft

staatlich

Ort

Krakau, Polen

Rektor

Jerzy Lis

Studierende

18,074^[1] (12/2023)

Mitarbeiter

4,167^[2] (31. Dezember 2017)

davon Professoren

157^[2] (31. Dezember 2017)

Netzwerke

IAU^[3]

Website

www.agh.edu.pl (<https://www.wagh.edu.pl/>)



Hauptgebäude der AGH



■ Field measurements

The AGH Environmental Physics Group is a leading national and internationally recognised center for environmental measurements, particularly using isotope methods. We present selected types of measurements performed by our group, along with references to relevant publications.



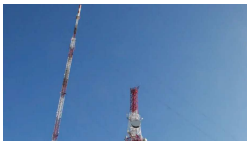
■ AGH KASLAB, Kasprowy Wierch (1989 m amsl)



■ AGH Campus 20m Tower (Kraków)



■ Hydrological and hydrogeological studies



■ Measurements at the 300-m BIK tower in North-East Poland



■ UAV-borne hydrological and meteorological observations



■ Meteo and air-quality profiling from a tethered balloon in Kraków

■ Modelling & software

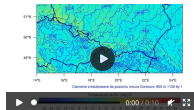
AGH Environmental Physics Group / Modelling & software

Computer simulations are an indispensable tool in our work. We specialize in the use of numerical models in environmental research, as well as in the engineering of open-source research software for use in environmental physics and, more broadly, in computational sciences. For high-performance computing, we employ AGH Cyfronet supercomputing resources. Methods and applications of numerical simulations and software engineering are part of our course offering. We leverage open-source software development and dissemination workflows for knowledge transfer across scientific disciplines and towards users outside of academia.

■ Operational weather prediction using WRF

contact: **Miroslaw Zimnoch**

Our WRF-based daily workflow is set up to compute 48-hour predictions on a country-wide domain, with initial and boundary conditions fetched from the NOAA GFS output. The forecasts are publicly available in form of weather maps and meteograms at meteo.fis.agh.edu.pl website.



■ Open-source research software engineering

contact: **open-atmos-krk team**

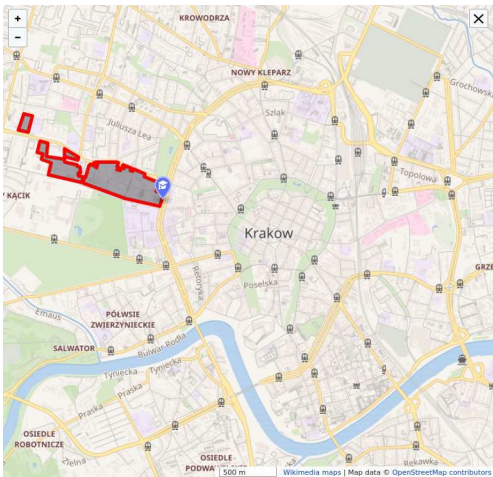
Our team, with collaborators from partnering institutions as well as AGH students, develops and maintains several open-source Python packages:

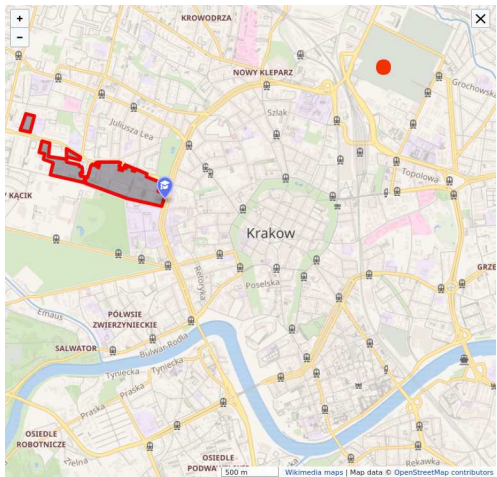
■ **numba-mpi** [pypi package 1.5.1](#) [Anaconda.org 1.5.1](#) [AUR package 1.5.1](#)

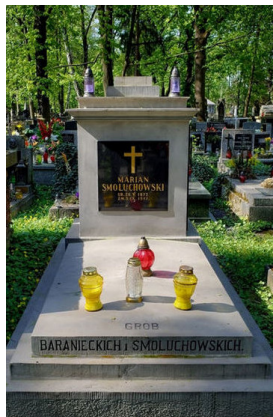
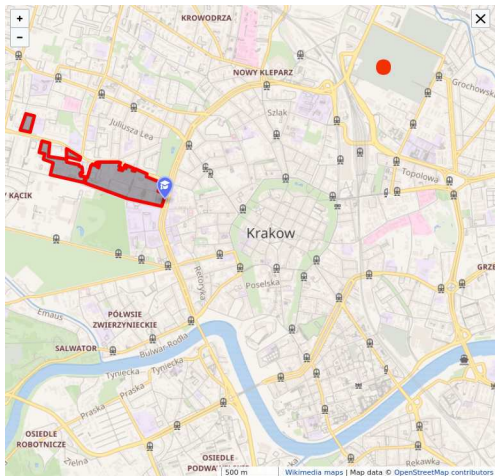


offers access to the Message Passing Interface (MPI) routines from Python code that uses the Numba just-in-time (JIT) compiler, enabling use of MPI communication in high-performance, multi-threaded, JIT-compiled Python code

use case: **py-pde** developed at [Max Planck Institute for Dynamics and](https://www.mpi-hd.mpg.de/)







Smoluchowski (1918) & Rudzki (1911)

Versuch einer mathematischen Theorie der Koagulationskinetik kolloider Lösungen.

Von

M. v. Smoluchowski.

(Mit 3 Figuren im Text.)

(Eingegangen am 8. 9. 16.)

I. Einleitung.

So sehr auch bis heute die Literatur über Koagulation kolloider Lösungen angewachsen ist, sind doch unsere Kenntnisse betreffs des quantitativen Verlaufs, sowie betreffs des Mechanismus des Koagulationsprozesses äusserst mangelhaft. Die meisten Forscher begnügen sich mit qualitativen Beobachtungen oder stellen ihre Messungsreihen in Tabellen oder Kurvenform¹⁾ dar, da die mathematische Wiedergabe derselben auf aussergewöhnliche Schwierigkeiten stösst.

In den interessanten Arbeiten²⁾ von S. Miyazawa, N. Ishizaka, H. Freundlich, J. A. Gann wird allerdings eine formelmässige Zusammenfassung des empirischen Versuchsmaterials, sowie eine Aufklärung desselben nach Analogie mit den Gesetzen der chemischen Kinetik angestrebt. Aber klare Gesetzmässigkeiten haben sich bisher auf diese Weise nicht ergeben, und wurden sogar gewisse, anfangs aufgestellte Gesetzformeln (Paine, Freundlich und Ishizaka) bei exakterer Prüfung (Freundlich und Gann) als unhaltbar zurückgenommen³⁾.

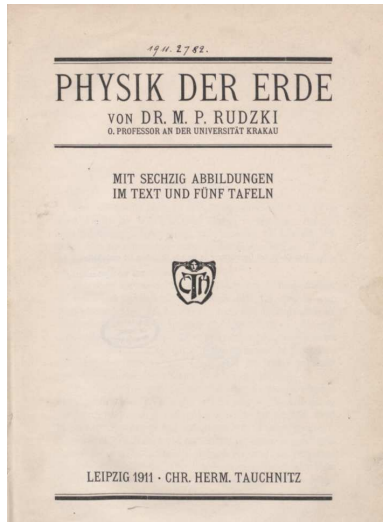
Die Erfolglosigkeit der bisherigen Versuche, auf dem empirisch-induktiven Wege zu einem Verständnis der hier geltenden Gesetze zu gelangen, kann man nun als einen Grund auffassen, einmal den um-

¹⁾ Vgl. z. B.: A. Galecki, Zeitschr. f. anorg. Chemie 74, 174 (1912); Kolloid-Zeitschr. 10, 169 (1912); A. Lottermoser, Kolloid-Zeitschr. 15, 145 (1914); H. H. Paine, Kolloidchem. Beihefte 4, 24 (1912); Kolloid-Zeitschr. 11, 115 (1912).

²⁾ S. Miyazawa, Journ. Chem. Soc. Tokio 33, 1179, 1210 (1912); N. Ishizaka, Zeitschr. f. physik. Chemie 83, 97 (1913); H. Freundlich u. N. Ishizaka, ebendort 85, 393 (1913); Kolloid-Zeitschr. 12, 230 (1913); J. Gann, Kolloidchem. Beihefte 8, 64 (1915).

³⁾ Siehe Abschnitt VI.

Zeitschrift f. physik. Chemie. XCII.



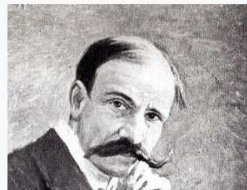
Maurycy Pius Rudzki (1862–1916)

Maurycy Pius Rudzki

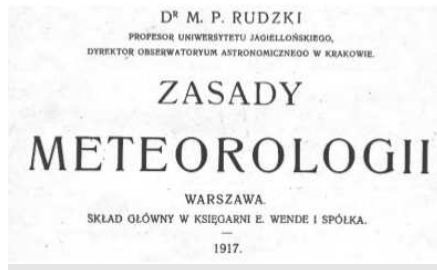
From Wikipedia, the free encyclopedia

Maurycy Pius Rudzki (b. 1862, d. 1916) was the first person to call himself a professor of geophysics. He held the Chair of Geophysics at the Jagiellonian University in Kraków, and established the Institute of Geophysics there in 1895. His research specialty was elastic anisotropy, as applied to wave propagation in the earth, and he established many of the fundamental results in that arena. ^[1]

Maurycy Pius Rudzki



“Principles of Meteorology” 1917



cloud modelling: fast-forward to 2020-ties



“Cloud and ship. Ukraine, Crimea, Black sea”

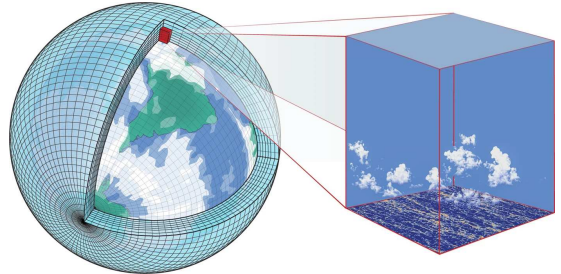
(photo: Yevgen Timashov / National Geographic)

cloud modelling: fast-forward to 2020-ties



“Cloud and ship. Ukraine, Crimea, Black sea”

(photo: Yevgen Timashov / National Geographic)



“Grid cells in a global climate model and a large-eddy simulation of shallow cumulus clouds at 5 m resolution”

(fig. from Schneider et al. 2017)



The screenshot shows the official website of the Intergovernmental Panel on Climate Change (IPCC). The page features a blue header with navigation links such as 'ABOUT', 'DATA', 'DOCUMENTATION', 'FOCAL POINTS PORTAL', 'BUREAU PORTAL', 'LIBRARY', 'LINKS', 'LANGUAGES', and a 'SEARCH' button. Below the header, the IPCC logo is prominently displayed on the left, while a horizontal menu on the right includes 'REPORTS', 'SYNTHESIS REPORT', 'WORKING GROUPS', 'ACTIVITIES', 'NEWS', 'FOLLOW', and 'SHARE'. The main content area is titled 'The Intergovernmental Panel on Climate Change' in large, bold, blue text. To the right of this title, a descriptive paragraph states: 'The Intergovernmental Panel on Climate Change (IPCC) is the United Nations body for assessing the science related to climate change.' A horizontal blue line separates the title from the content below. At the bottom of the page, a grey box on the left identifies the 'WORKING GROUP II SIXTH ASSESSMENT REPORT'. On the right, logos for the WHO, UNEP, and the Nobel Peace Prize are displayed, indicating the report's status as a Nobel Prize-winning work.

MENU

ABOUT DATA DOCUMENTATION FOCAL POINTS PORTAL BUREAU PORTAL LIBRARY LINKS LANGUAGES ▼ SEARCH

ipcc

REPORTS SYNTHESIS REPORT WORKING GROUPS ACTIVITIES NEWS

CALENDAR

FOLLOW SHARE

The Intergovernmental Panel on Climate Change

The Intergovernmental Panel on Climate Change (IPCC) is the United Nations body for assessing the science related to climate change.

WORKING GROUP II SIXTH ASSESSMENT REPORT

WHO UNEP Nobel 2007 PEACE PRIZE FOR THE NIPCC FOUNDATION

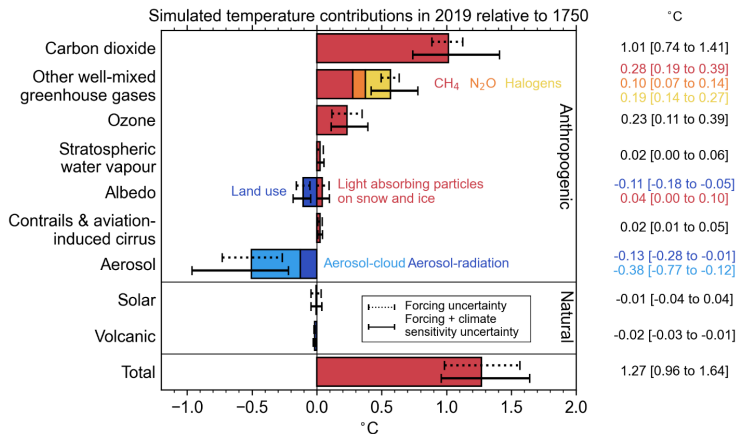


Figure 7.7: The contribution of forcing agents to 2019 temperature change relative to 1750 produced using the two-layer emulator (Supplementary Material 7.SM.2), constrained to assessed ranges for key climate metrics described in Cross-Chapter Box 7.1.

- aerosol-cloud interactions
modelling in (pure) Python?
-



PyMPDATA



PySDM



Numba-MPI

- **PyMPDATA**

MPDATA key concepts: UPWIND discretisation

- advection equation / scalar conservation law:

$$\partial_t(G\psi) + \nabla \cdot (\mathbf{v}\psi) = GR$$

$\psi(\mathbf{x}, t)$: advected scalar field (advectee),

$\mathbf{v} = \{u, \dots\} = G\dot{\mathbf{x}}$: flow velocity vector field (advectee),

$G(\mathbf{x})$: fluid density, Jacobian of coordinate transformation, or their product

MPDATA key concepts: UPWIND discretisation

- advection equation / scalar conservation law:

$$\partial_t(G\psi) + \nabla \cdot (\mathbf{v}\psi) = GR$$

$\psi(\mathbf{x}, t)$: advected scalar field (advectee),

$\mathbf{v} = \{u, \dots\} = G\dot{\mathbf{x}}$: flow velocity vector field (advector),

$G(\mathbf{x})$: fluid density, Jacobian of coordinate transformation, or their product

- UPWIND discretisation on a staggered grid (n numbers time steps, i grid steps):

$$\frac{\psi_i^{n+1} - \psi_i^n}{\Delta t} + \frac{\overbrace{f(\psi_i^n, \psi_{i+1}^n, u_{i+1/2}^n)}^{\text{right-hand wall flux}} - \overbrace{f(\psi_{i-1}^n, \psi_i^n, u_{i-1/2}^n)}^{\text{left-hand wall flux}}}{\Delta x} = 0$$

MPDATA key concepts: UPWIND discretisation

- advection equation / scalar conservation law:

$$\partial_t(G\psi) + \nabla \cdot (\mathbf{v}\psi) = GR$$

$\psi(\mathbf{x}, t)$: advected scalar field (advectee),

$\mathbf{v} = \{u, \dots\} = G\dot{\mathbf{x}}$: flow velocity vector field (advector),

$G(\mathbf{x})$: fluid density, Jacobian of coordinate transformation, or their product

- UPWIND discretisation on a staggered grid (n numbers time steps, i grid steps):

$$\frac{\psi_i^{n+1} - \psi_i^n}{\Delta t} + \frac{\overbrace{f(\psi_i^n, \psi_{i+1}^n, u_{i+1/2}^n)}^{\text{right-hand wall flux}} - \overbrace{f(\psi_{i-1}^n, \psi_i^n, u_{i-1/2}^n)}^{\text{left-hand wall flux}}}{\Delta x} = 0$$

$$\partial_t \psi|_i^n + u \partial_x \psi|_i^n = \underbrace{\left(|u| \frac{\Delta x}{2} - u^2 \frac{\Delta t}{2} \right)}_{k - \text{numerical diffusion}} \partial_x^2 \psi|_i^n$$

(e.g., Roberts & Weiss 1966, doi:10.2307/2003507)

MPDATA key concepts: antidiffusive pseudo-velocities

- diffusion can be cast as advection with a pseudo-velocity:

$$\partial_t \psi = k \partial_x^2 \psi + \dots \quad \rightsquigarrow \quad \partial_t \psi = \partial_x \left(\underbrace{k \frac{\partial_x \psi}{\psi}}_{\text{pseudo-velocity}} \psi \right) + \dots$$

(e.g., Lange 1973, doi:10.2172/4308175)

MPDATA key concepts: antidiffusive pseudo-velocities

- diffusion can be cast as advection with a pseudo-velocity:

$$\partial_t \psi = k \partial_x^2 \psi + \dots \quad \rightsquigarrow \quad \partial_t \psi = \partial_x \underbrace{\left(k \frac{\partial_x \psi}{\psi} \psi \right)}_{\text{pseudo-velocity}} + \dots$$

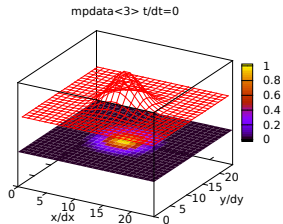
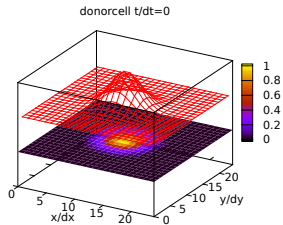
(e.g., Lange 1973, doi:10.2172/4308175)

- “Smolarkiewicz algorithm” (MPDATA): upwind-integrate backwards-in-time, with an anti-diffusive pseudo velocity to reverse the effects of numerical diffusion, iteratively (m numbers iteration)

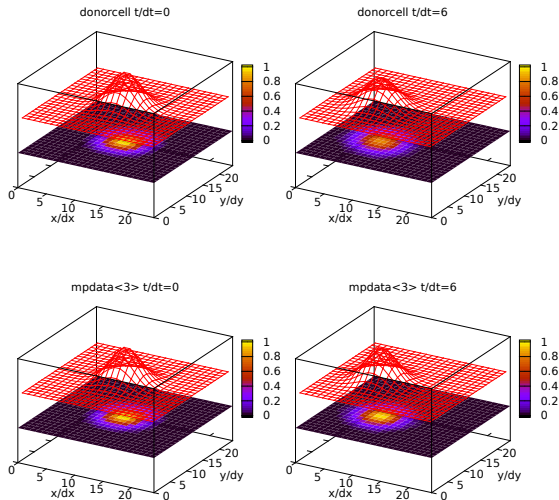
$$C_{i-1/2}^{m+1} = \frac{\Delta t}{\Delta x} k_{i-1/2}^m \left. \frac{\partial_x \psi}{\psi} \right|_{i-1/2}^m \approx \begin{cases} 0 & \text{if } \psi_i^m + \psi_{i-1}^m = 0 \\ \left[|C_{i-1/2}^m| - (C_{i-1/2}^m)^2 \right] \frac{\psi_i^m - \psi_{i-1}^m}{\psi_i^m + \psi_{i-1}^m} & \text{otherwise} \end{cases}$$

(Smolarkiewicz 1983 MWR, 1984 JCP: doi:10.1016/0021-9991(84)90121-9)

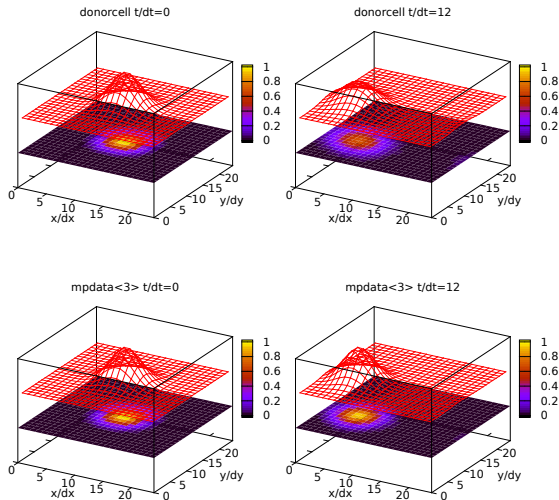
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATA's)



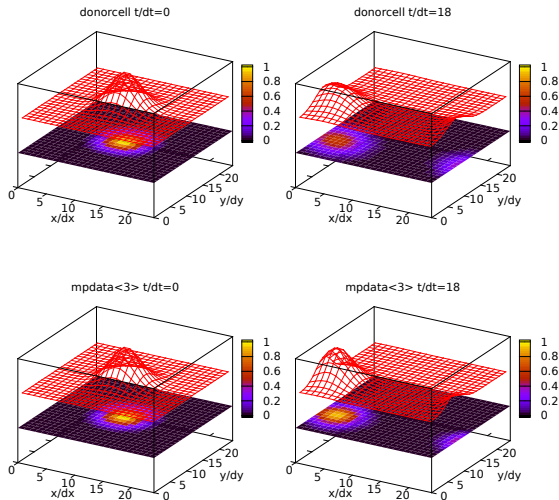
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



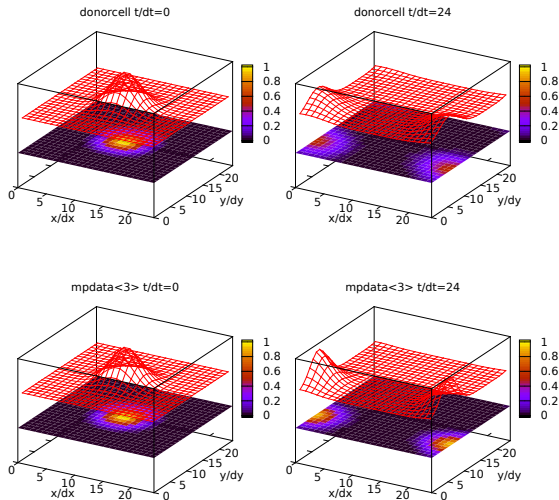
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



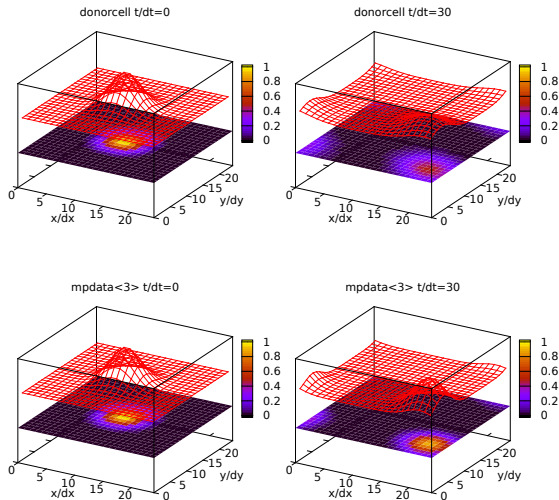
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



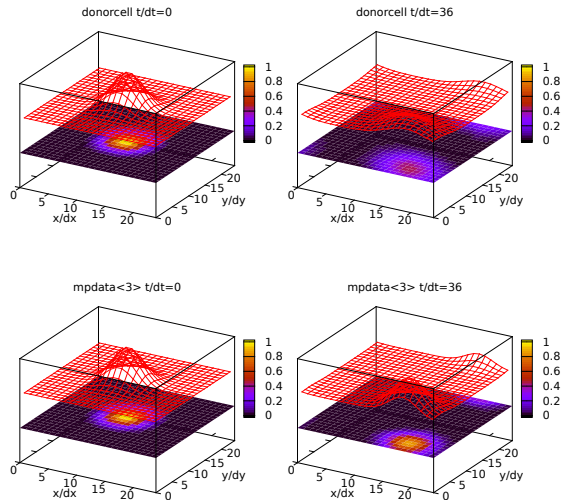
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



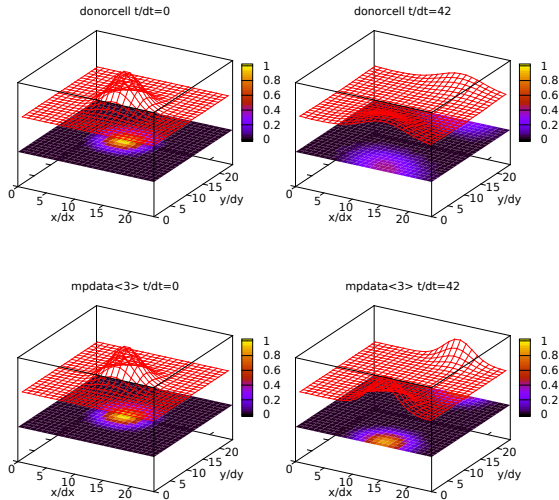
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



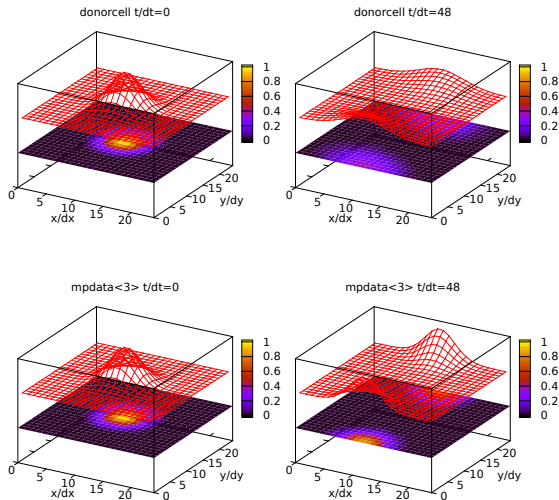
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



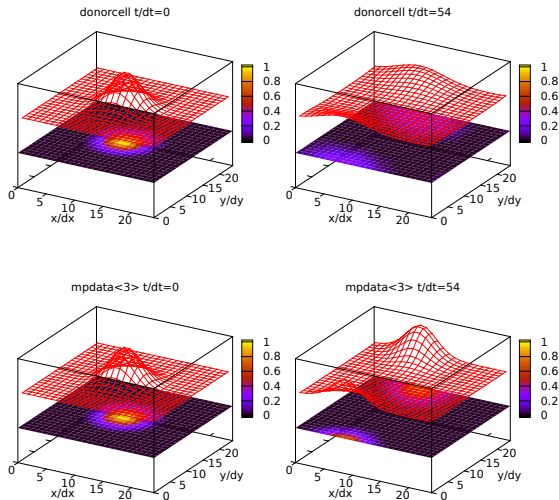
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



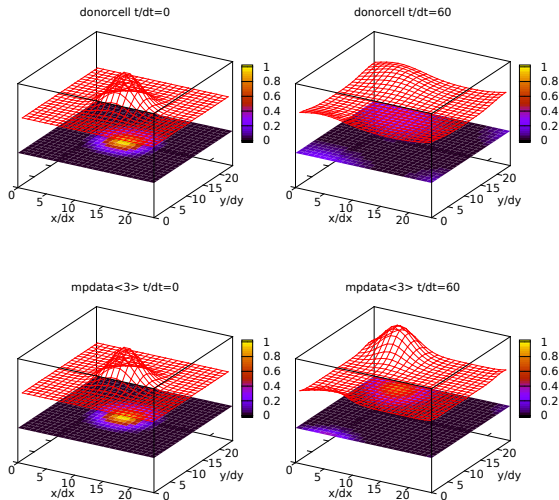
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



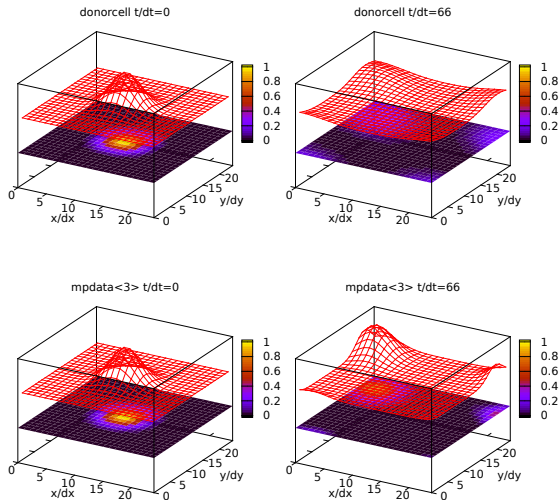
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



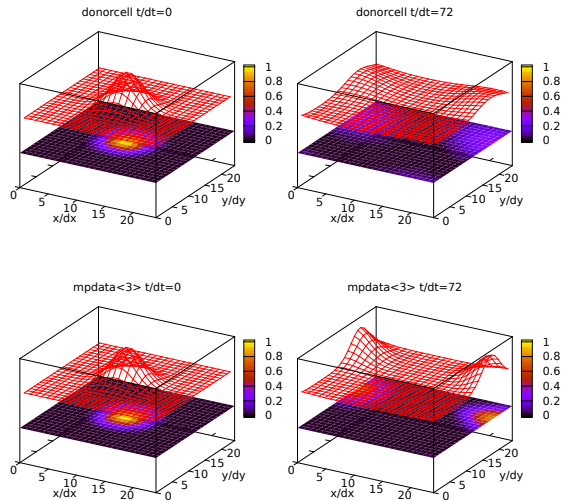
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



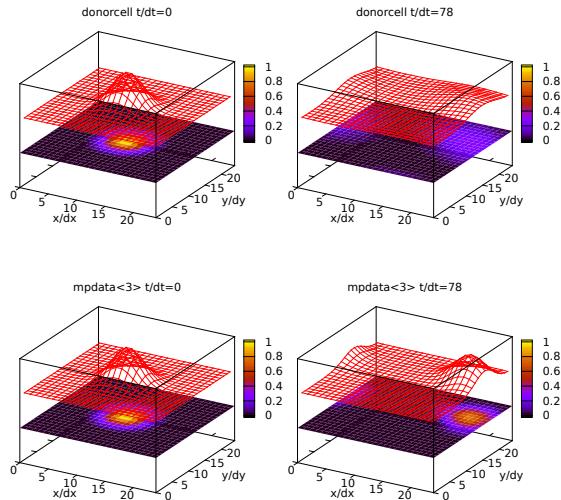
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



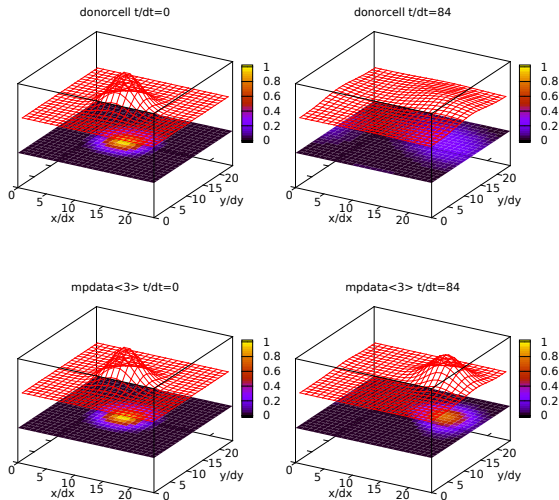
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



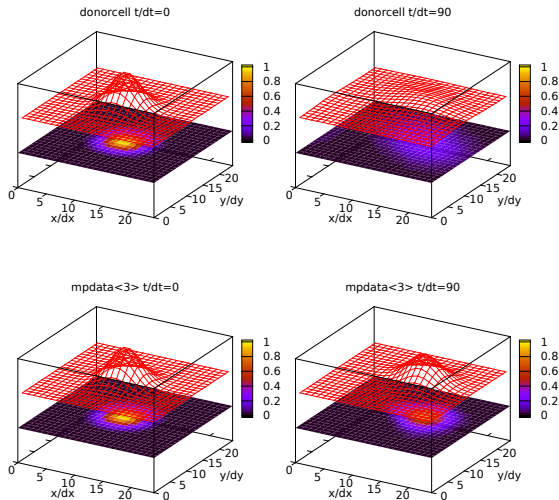
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



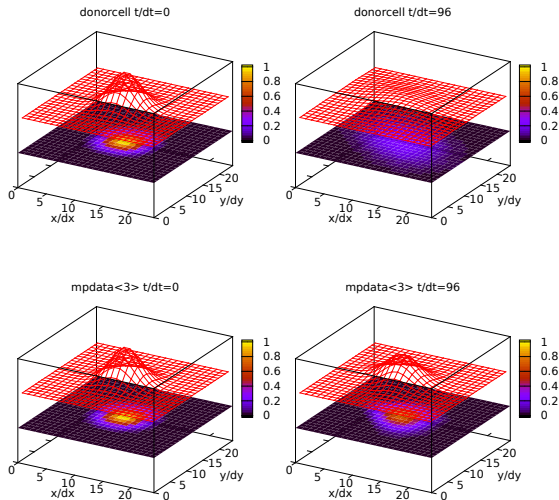
MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



MPDATA: "M" for multi-dimensional (2D is not a sum of two 1D MPDATAs)



MPDATA variants (structured grid)

- basic (+iterations): Smolarkiewicz 1983

MPDATA variants (structured grid)

- basic (+iterations): Smolarkiewicz 1983
- n-dimensions, diffusion, divergent flow corrections: Smolarkiewicz 1984

MPDATA variants (structured grid)

- basic (+iterations): Smolarkiewicz 1983
- n-dimensions, diffusion, divergent flow corrections: Smolarkiewicz 1984
- coordinate transformation: Smolarkiewicz and Clark 1986, Smolarkiewicz and Margolin 1993

MPDATA variants (structured grid)

- basic (+iterations): Smolarkiewicz 1983
- n-dimensions, diffusion, divergent flow corrections: Smolarkiewicz 1984
- coordinate transformation: Smolarkiewicz and Clark 1986, Smolarkiewicz and Margolin 1993
- resursive anti-diffusive velocities: Beason & Margolin 1988

MPDATA variants (structured grid)

- basic (+iterations): Smolarkiewicz 1983
- n-dimensions, diffusion, divergent flow corrections: Smolarkiewicz 1984
- coordinate transformation: Smolarkiewicz and Clark 1986, Smolarkiewicz and Margolin 1993
- resursive anti-diffusive velocities: Beason & Margolin 1988
- flux-corrected transport: Smolarkiewicz and Grabowski 1990

MPDATA variants (structured grid)

- basic (+iterations): Smolarkiewicz 1983
- n-dimensions, diffusion, divergent flow corrections: Smolarkiewicz 1984
- coordinate transformation: Smolarkiewicz and Clark 1986, Smolarkiewicz and Margolin 1993
- resursive anti-diffusive velocities: Beason & Margolin 1988
- flux-corrected transport: Smolarkiewicz and Grabowski 1990
- third-order terms: Smolarkiewicz and Margolin 1998

MPDATA variants (structured grid)

- basic (+iterations): Smolarkiewicz 1983
- n-dimensions, diffusion, divergent flow corrections: Smolarkiewicz 1984
- coordinate transformation: Smolarkiewicz and Clark 1986, Smolarkiewicz and Margolin 1993
- resursive anti-diffusive velocities: Beason & Margolin 1988
- flux-corrected transport: Smolarkiewicz and Grabowski 1990
- third-order terms: Smolarkiewicz and Margolin 1998
- infinite-gauge variant: Smolarkiewicz 2006

MPDATA variants (structured grid)

- basic (+iterations): Smolarkiewicz 1983
- n-dimensions, diffusion, divergent flow corrections: Smolarkiewicz 1984
- coordinate transformation: Smolarkiewicz and Clark 1986, Smolarkiewicz and Margolin 1993
- resursive anti-diffusive velocities: Beason & Margolin 1988
- flux-corrected transport: Smolarkiewicz and Grabowski 1990
- third-order terms: Smolarkiewicz and Margolin 1998
- infinite-gauge variant: Smolarkiewicz 2006
- fully third-order variant: Waruszewski et al. 2018

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
-----------	--------------------	----	--	------------

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
ROMS	FORTTRAN	3D	 /myroms	UCLA (?)

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
ROMS	FORTTRAN	3D	 /myroms	UCLA (?)
PISM	C++	2D	 /pism	U. Alaska Fairbanks

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
ROMS	FORTTRAN	3D	 /myroms	UCLA (?)
PISM	C++	2D	 /pism	U. Alaska Fairbanks
babyEULAG	FORTTRAN	2D	 /igfuw/bE_SDs	NCAR

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
ROMS	FORTTRAN	3D	 /myroms	UCLA (?)
PISM	C++	2D	 /pism	U. Alaska Fairbanks
babyEULAG	FORTTRAN	2D	 /igfuw/bE_SDs	NCAR
apc-llc/mpdata	C/CUDA	3D	 /apc-llc	Rus. Acad. Sci.

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
ROMS	FORTTRAN	3D	 /myroms	UCLA (?)
PISM	C++	2D	 /pism	U. Alaska Fairbanks
babyEULAG	FORTTRAN	2D	 /igfuw/bE_SDs	NCAR
apc-llc/mpdata	C/CUDA	3D	 /apc-llc	Rus. Acad. Sci.
ICAR	FORTTRAN	3D	 /NCAR/icar	NCAR

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
ROMS	FORTTRAN	3D	 /myroms	UCLA (?)
PISM	C++	2D	 /pism	U. Alaska Fairbanks
babyEULAG	FORTTRAN	2D	 /igfuw/bE_SDs	NCAR
apc-llc/mpdata	C/CUDA	3D	 /apc-llc	Rus. Acad. Sci.
ICAR	FORTTRAN	3D	 /NCAR/icar	NCAR
sbPOM	FORTTRAN	2D	 /RinceWND/extPOM	Stony Brook

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
ROMS	FORTTRAN	3D	 /myroms	UCLA (?)
PISM	C++	2D	 /pism	U. Alaska Fairbanks
babyEULAG	FORTTRAN	2D	 /igfuw/bE_SDs	NCAR
apc-llc/mpdata	C/CUDA	3D	 /apc-llc	Rus. Acad. Sci.
ICAR	FORTTRAN	3D	 /NCAR/icar	NCAR
sbPOM	FORTTRAN	2D	 /RinceWND/extPOM	Stony Brook

reusable:

libmpdata++	C++/Blitz++	1,2,3D	 /igfuw	UWarsaw
-------------	-------------	--------	--	---------

MPDATA implementations

known closed-source:

- NCAR EULAG
- COSMO (alternative dynamical core)
- ECMWF IFS-FVM

open-source:

integrated into CFD packages:

AtmosFOAM	C++ & Python/Numba	3D	 /AtmosFOAM	U. Reading
ROMS	FORTTRAN	3D	 /myroms	UCLA (?)
PISM	C++	2D	 /pism	U. Alaska Fairbanks
babyEULAG	FORTTRAN	2D	 /igfuw/bE_SDs	NCAR
apc-llc/mpdata	C/CUDA	3D	 /apc-llc	Rus. Acad. Sci.
ICAR	FORTTRAN	3D	 /NCAR/icar	NCAR
sbPOM	FORTTRAN	2D	 /RinceWND/extPOM	Stony Brook

reusable:

libmpdata++	C++/Blitz++	1,2,3D	 /igfuw	UWarsaw
PyMPDATA	Python/Numba	1,2,3D	 /open-atmos	UJ/AGH.edu.pl

PyMPDATA: design goals, tech stack, features

- Numba JIT $\rightsquigarrow$ pure-Python code with compiled-language performance (plus OpenMP-like multi-threading)



PyMPDATA: 100% Python codebase

```
@numba.njit(**options.jit_flags)
def a_term(psi):
    """eq. 13 in [Smolarkiewicz 1984](https://doi.org/10.1016/0021-9991(84)90121-9);
    eq. 17a in [Smolarkiewicz & Margolin 1998](https://doi.org/10.1006/jcph.1998.5901)"""
    result = ats(*psi, 1) - ats(*psi, 0)
    if infinite_gauge:
        return result / 2
    return result / (ats(*psi, 1) + ats(*psi, 0) + epsilon)

@numba.njit(**options.jit_flags)
def b_term(psi):
    """eq. 13 in [Smolarkiewicz 1984](https://doi.org/10.1016/0021-9991(84)90121-9);
    eq. 17b in [Smolarkiewicz & Margolin 1998](https://doi.org/10.1006/jcph.1998.5901)"""
    result = ats(*psi, 1, 1) + ats(*psi, 0, 1) - ats(*psi, 1, -1) - ats(*psi, 0, -1)
    if infinite_gauge:
        return result / 4

    return result / (
        ats(*psi, 1, 1)
        + ats(*psi, 0, 1)
        + ats(*psi, 1, -1)
        + ats(*psi, 0, -1)
        + epsilon
    )
```

PyMPDATA: design goals, tech stack, features

- Numba JIT $\rightsquigarrow$ pure-Python code with compiled-language performance (plus OpenMP-like multi-threading)
- runs on Linux, macOS & Windows (no compilation, just "pip install")



PyMPDATA

PyMPDATA: design goals, tech stack, features



PyMPDATA

- Numba JIT $\rightsquigarrow$ pure-Python code with compiled-language performance (plus OpenMP-like multi-threading)
- runs on Linux, macOS & Windows (no compilation, just "pip install")
- > 90% unit-test coverage (codecov) and growing...

PyMPDATA: design goals, tech stack, features



- Numba JIT $\rightsquigarrow$ pure-Python code with compiled-language performance (plus OpenMP-like multi-threading)
- runs on Linux, macOS & Windows (no compilation, just "pip install")
- > 90% unit-test coverage (codecov) and growing...
- API: 5 classes & single "advance" method

PyMPDATA: design goals, tech stack, features



PyMPDATA

- Numba JIT $\rightsquigarrow$ pure-Python code with compiled-language performance (plus OpenMP-like multi-threading)
- runs on Linux, macOS & Windows (no compilation, just "pip install")
- > 90% unit-test coverage (codecov) and growing...
- API: 5 classes & single "advance" method
- array-traversal abstractions avoiding explicit fill-halo or barrier calls

PyMPDATA: design goals, tech stack, features



- Numba JIT $\rightsquigarrow$ pure-Python code with compiled-language performance (plus OpenMP-like multi-threading)
- runs on Linux, macOS & Windows (no compilation, just "pip install")
- > 90% unit-test coverage (codecov) and growing...
- API: 5 classes & single "advance" method
- array-traversal abstractions avoiding explicit fill-halo or barrier calls
- docs (and CI) covers usage from Python, Julia, Matlab and Rust

PyMPDATA: design goals, tech stack, features



- Numba JIT $\rightsquigarrow$ pure-Python code with compiled-language performance (plus OpenMP-like multi-threading)
- runs on Linux, macOS & Windows (no compilation, just "pip install")
- > 90% unit-test coverage (codecov) and growing...
- API: 5 classes & single "advance" method
- array-traversal abstractions avoiding explicit fill-halo or barrier calls
- docs (and CI) covers usage from Python, Julia, Matlab and Rust
- suite of 20+ Jupyter notebook examples maintained with the project all with badges enabling **single-click execution on Colab**

PyMPDATA: design goals, tech stack, features



- Numba JIT $\rightsquigarrow$ pure-Python code with compiled-language performance (plus OpenMP-like multi-threading)
- runs on Linux, macOS & Windows (no compilation, just "pip install")
- > 90% unit-test coverage (codecov) and growing...
- API: 5 classes & single "advance" method
- array-traversal abstractions avoiding explicit fill-halo or barrier calls
- docs (and CI) covers usage from Python, Julia, Matlab and Rust
- suite of 20+ Jupyter notebook examples maintained with the project all with badges enabling **single-click execution on Colab**
- examples in 1D, 2D & 3D: advection-diffusion, bin cloud μ -physics, spherical coordinates, shallow-water, Black-Scholes, Burgers, Boussinesq



Documentation

What is PyMPDATA?

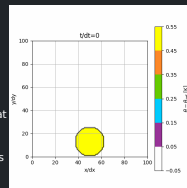
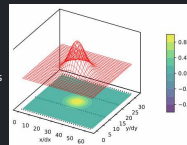
PyMPDATA is a **Numba-accelerated** **multi-threaded** Pythonic implementation of the **MPDATA algorithm** of Smolarkiewicz et al. used in geophysical fluid dynamics and beyond for **numerically solving generalised convection-diffusion PDEs**. PyMPDATA supports integration in 1D, 2D and 3D structured meshes with optional coordinate transformations. The first animation shown depicts a "hello-world" 2D advection-only simulation with dotted lines indicating **domain decomposition** across three threads. The second animation depicts an MPDATA solution to coupled mass and momentum conservation equations for a buoyancy-driven flow in Boussinesq approximation (see Jaruga et al. 2015 example).

A separate project called **PyMPDATA-MPI** depicts how **numba-mpi** can be used to enable **distributed memory parallelism** in PyMPDATA.

What is the difference between PyMPDATA and PyMPDATA-examples?

PyMPDATA is a Python package that provides the MPDATA algorithm implementation. It is a library that can be used in your own projects.

PyMPDATA-examples is a Python package that provides examples of how to use PyMPDATA. It includes common Python modules used in PyMPDATA smoke tests and in example Jupyter notebooks (but the package wheels do not include the notebooks, only .py files imported from the notebooks and PyMPDATA tests).



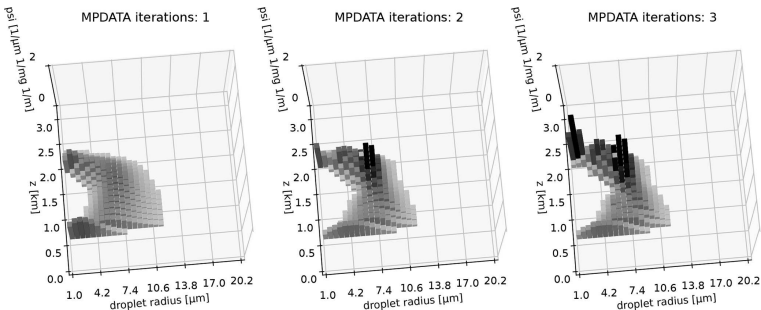
MPDATA for condensational growth in bin μ -physics (Olesik et al. 2022)

Geosci. Model Dev., 15, 3879–3899, 2022
<https://doi.org/10.5194/gmd-15-3879-2022>



On numerical broadening of particle-size spectra: a condensational growth study using PyMPDATA 1.0

Michael A. Olesik¹, Jakub Banaśkiewicz², Piotr Bartman², Manuel Baumgartner^{3,4}, Simon Unterstrasser⁵, and Sylwester Arabas^{6,2}



- spectro-spatial advection (single-column model)
- spectral broadening vs. MPDATA options

MPDATA for Asian option pricing using 2D PDE (Magnuszewski et al. 2025)

arXiv > q-fin > arXiv:2505.24435

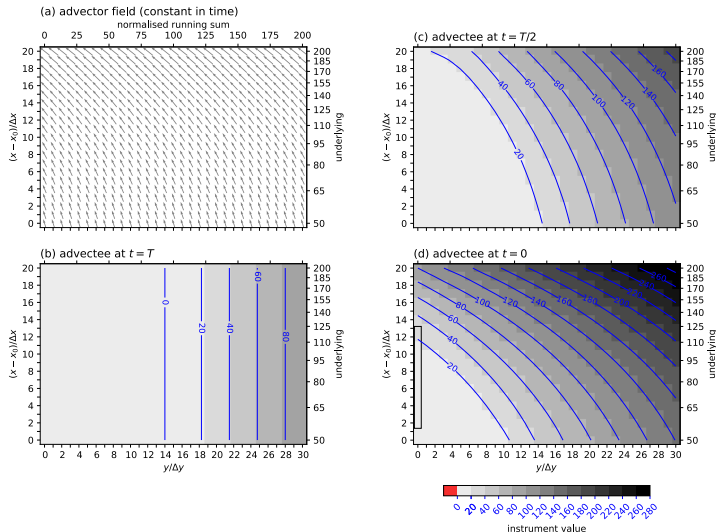
Quantitative Finance > Computational Finance

[Submitted on 30 May 2025]

Path-dependent option pricing with two-dimensional PDE using MPDATA

Paweł Magnuszewski, Sylwester Arabas

In this paper, we discuss a simple yet robust PDE method for evaluating path-dependent Asian-style options using the non-oscillatory forward-in-time second-order MPDATA finite-difference scheme. The valuation methodology involves casting the Black-Merton-Scholes equation as a transport problem by first transforming it into a homogeneous advection-diffusion PDE via variable substitution, and then expressing the diffusion term as an advective flux using the pseudo-velocity technique.





DOI: [10.21105/joss.03896](https://doi.org/10.21105/joss.03896)

Software

- [Review](#) ↗
- [Repository](#) ↗
- [Archive](#) ↗

Editor: [Arfon Smith](#) ↗

Reviewers:

- [@Chil](#)
- [@wdeconinck](#)

Submitted: 25 October 2021

Published: 05 September 2022

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

PyMPDATA v1: Numba-accelerated implementation of MPDATA with examples in Python, Julia and Matlab

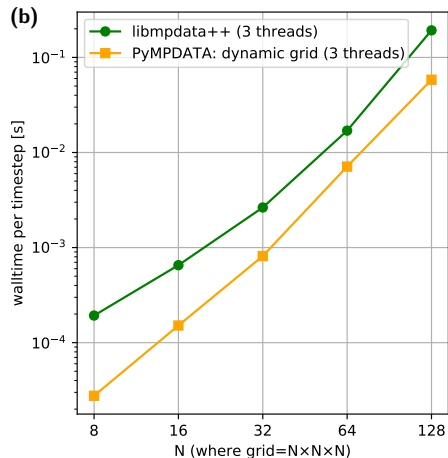
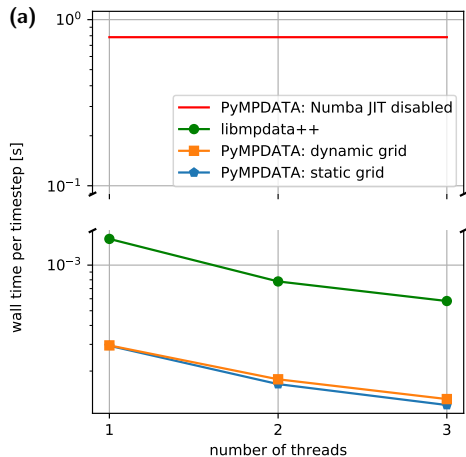
Piotr Bartman¹, Jakub Banaśkiewicz¹, Szymon Drenda¹, Maciej Manna¹, Michael A. Olesik¹, Paweł Rozwoda¹, Michał Sadowski¹, and Sylwester Arabas^{1,2}

¹ Jagiellonian University, Kraków, Poland ² University of Illinois at Urbana-Champaign, IL, USA

Statement of need

Convection-diffusion problems arise across a wide range of pure and applied research, in particular in geosciences, aerospace engineering, and financial modelling (for an overview of applications, see, e.g., section 1.1 in Morton (1996)). One of the key challenges in numerical solutions of problems involving advective transport is sign preservation of the advected field (for an overview of this and other aspects of numerical solutions to advection problems, see, e.g., Røed (2019)). The Multidimensional Positive Definite Advection Transport Algorithm (MPDATA) is a robust, explicit-in-time, and sign-preserving solver introduced in Smolarkiewicz (1983) and Smolarkiewicz (1984). MPDATA has been subsequently developed into a family of numerical schemes with numerous variants and solution procedures addressing a diverse set of problems in geophysical fluid dynamics and beyond. For reviews of MPDATA applications and variants, see, e.g., Smolarkiewicz & Margolin (1998) and Smolarkiewicz (2006).

Numba JIT & multi-threading: PyMPDATA vs. libmpdata++ performance



Bartman et al. 2022 (JOSS, doi:10.21105/joss.03896)

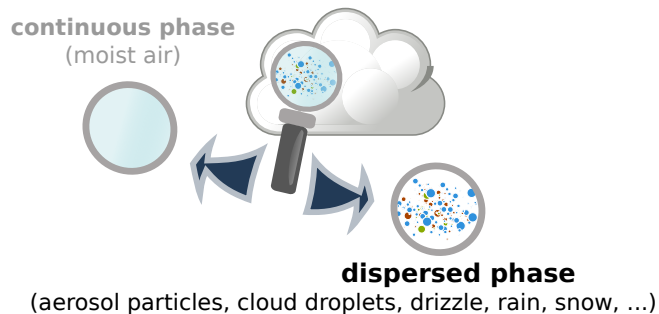
- **PySDM**



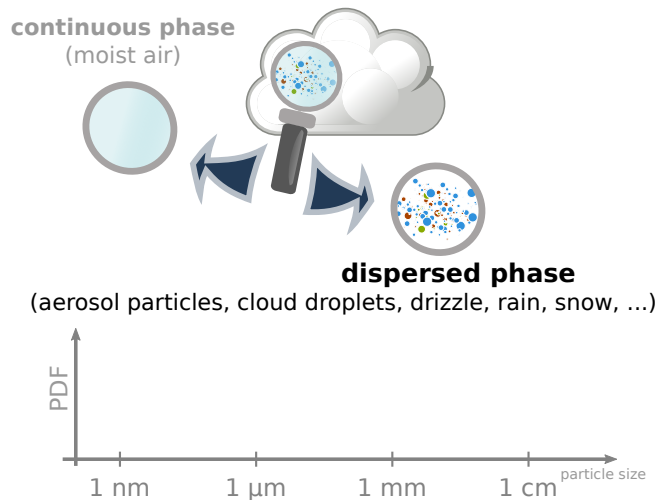
Eulerian vs. Lagrangian cloud microphysics



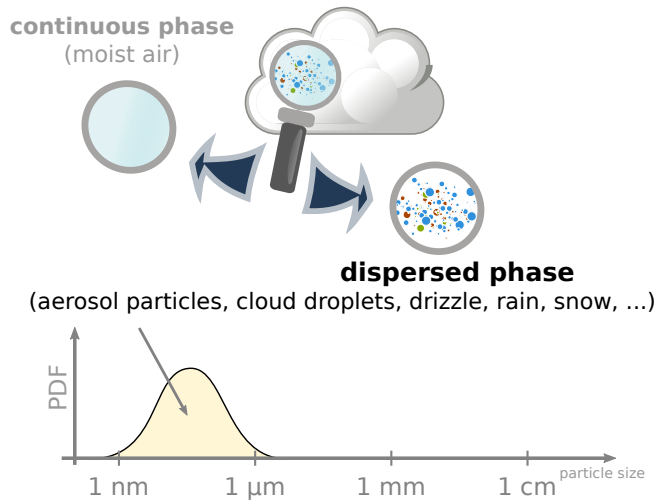
Eulerian vs. Lagrangian cloud microphysics



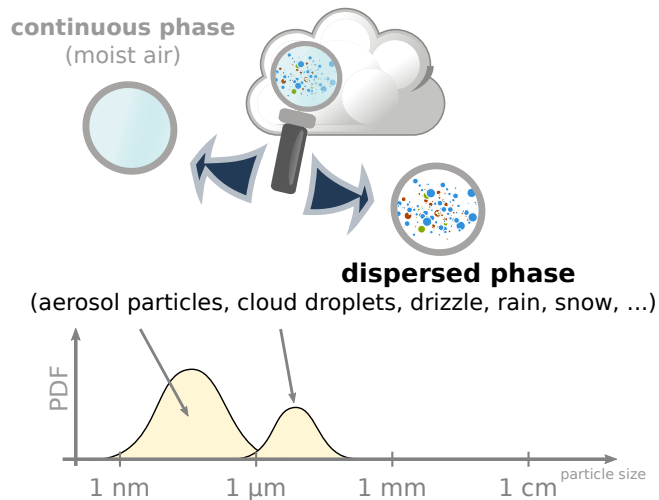
Eulerian vs. Lagrangian cloud microphysics



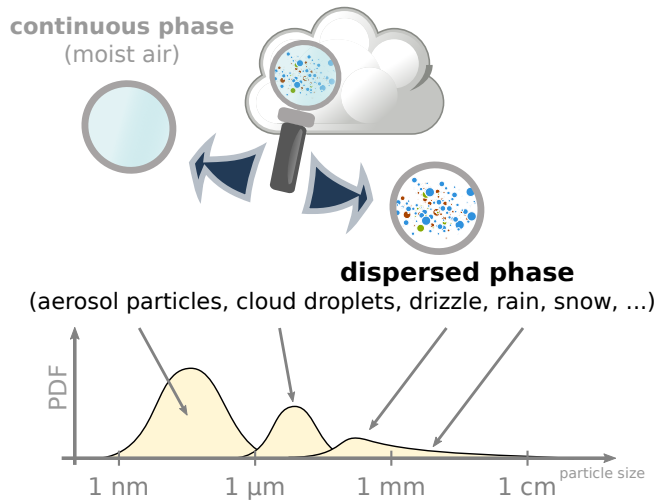
Eulerian vs. Lagrangian cloud microphysics



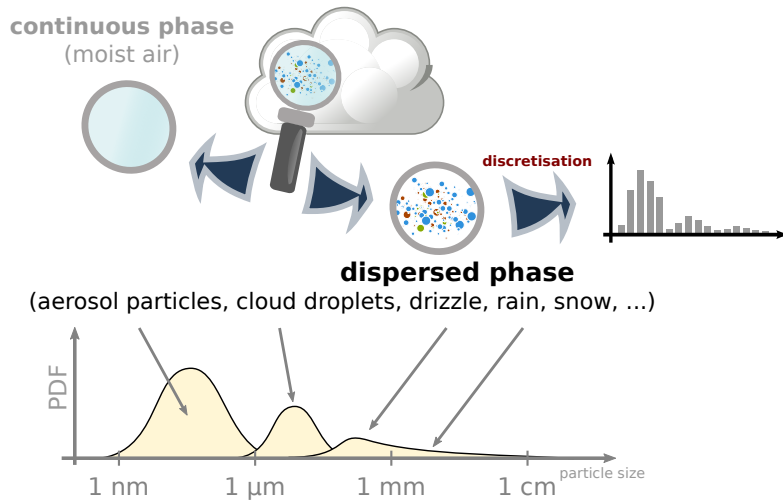
Eulerian vs. Lagrangian cloud microphysics



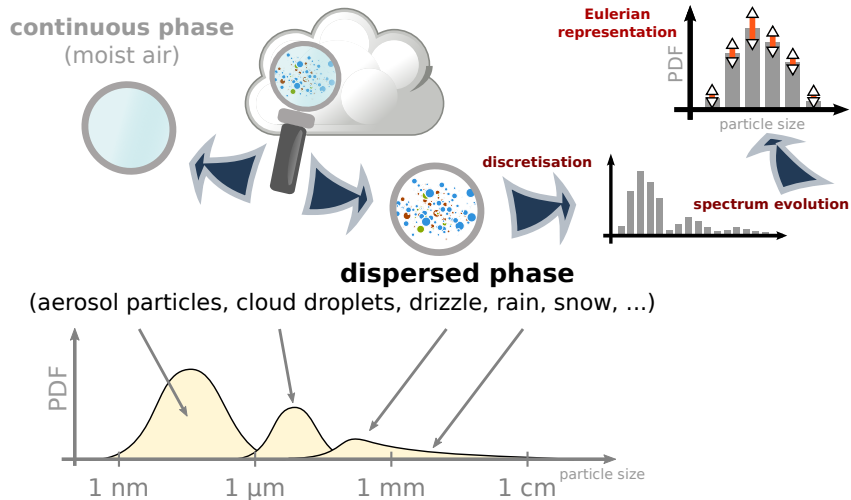
Eulerian vs. Lagrangian cloud microphysics



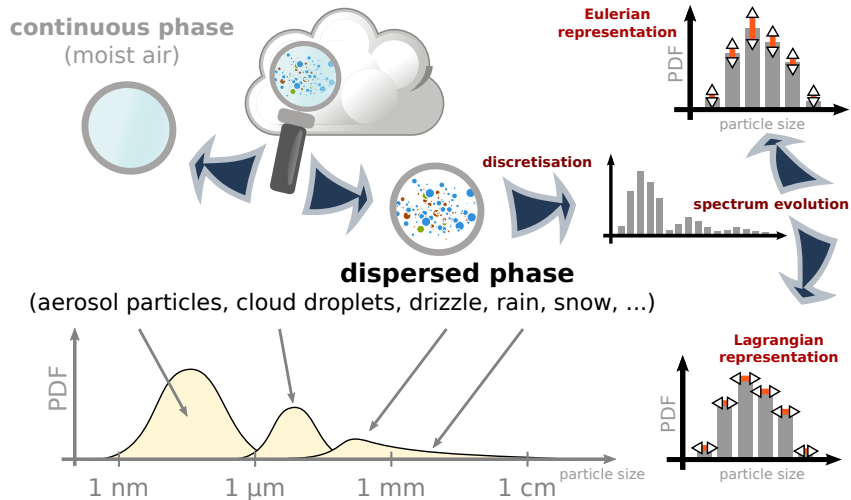
Eulerian vs. Lagrangian cloud microphysics



Eulerian vs. Lagrangian cloud microphysics



Eulerian vs. Lagrangian cloud microphysics



PDEs

ODEs

Eulerian vs. Lagrangian microphysics: a (probabilistic) breakthrough

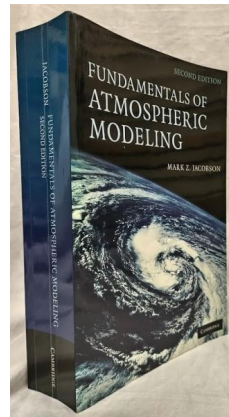
pre-2009:

„advantage of the full-moving size structure is that core particle material is preserved during growth ... second advantage ... it eliminates numerical diffusion ... [but] nucleation, coagulation ... cause problems ... the full-moving structure is not used in three-dimensional models”^a

„the use of a fixed grid allows for an easy implementation of collision processes, which is not possible for a moving grid (Lagrangian) approach”^b

^a Jacobson 2005: Fundamentals of Atmospheric Modeling

^b Simmel & Wurzel 2006: Condensation and activation in sectional cloud microphysical models



Eulerian vs. Lagrangian microphysics: a (probabilistic) breakthrough

pre-2009:

„advantage of the full-moving size structure is that core particle material is preserved during growth ... second advantage ... it eliminates numerical diffusion ... [but] nucleation, coagulation ... cause problems ... the full-moving structure is not used in three-dimensional models”^a

„the use of a fixed grid allows for an easy implementation of collision processes, which is not possible for a moving grid (Lagrangian) approach”^b

Shima 2009: Monte-Carlo particle-based collision algorithm for cloud simulations

^a Jacobson 2005: Fundamentals of Atmospheric Modeling

^b Simmel & Wurzler 2006: Condensation and activation in sectional cloud microphysical models

Eulerian vs. Lagrangian microphysics: a (probabilistic) breakthrough

pre-2009:

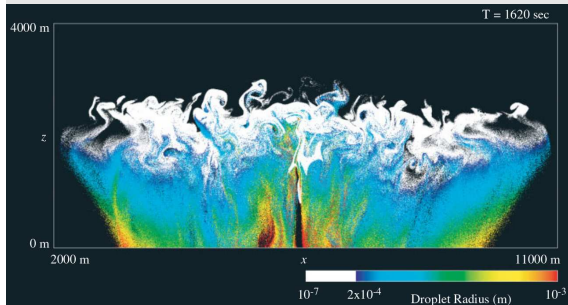
„advantage of the full-moving size structure is that core particle material is preserved during growth ... second advantage ... it eliminates numerical diffusion ... [but] nucleation, coagulation ... cause problems ... the full-moving structure is not used in three-dimensional models”^a

„the use of a fixed grid allows for an easy implementation of collision processes, which is not possible for a moving grid (Lagrangian) approach”^b

^a Jacobson 2005: Fundamentals of Atmospheric Modeling

^b Simmel & Wurzler 2006: Condensation and activation in sectional cloud microphysical models

Shima 2009: Monte-Carlo particle-based collision algorithm for cloud simulations



Super-droplet simulation of a shallow convective cloud
(figure: Shima et al. 2009, QJRM)

[Help](#) [Docs](#) [Sponsors](#) [Login](#) [Register](#)

pysdm 2.131

[pip install pysdm](#)

Released: Jul 3, 2025

Pythonic particle-based (super-droplet) warm-rain/aqueous-chemistry cloud microphysics package with box, parcel & 1D/2D prescribed-flow examples in Python, Julia and Matlab

Navigation

[Project description](#)[Release history](#)[Download files](#)

Verified details

These details have been [verified by PyPI](#)

Project links

[Documentation](#)[Source](#)[Tracker](#)

GitHub Statistics

[Repository](#)

★ Stars: 88

🔱 Forks: 54

🔍 Open issues: 201

🔧 Open PRs: 40

Maintainers



AgnieszkaZab

priebman

slayoo

Project description



PySDM

Python 3.8+ LLVM 16+ CUDA 11.5+ Thrust 1.21+ Julia 1.10+ macOS 10.15+ Windows 10+ Docker 20.10+ Maccos 10.15+
Open MPI 4.0.1+ PyTorch 1.12.0+ PyVista 0.29.0+ PyGMT 6.0.0+ PyGMT 6.0.0+ PyGMT 6.0.0+
EU Funding by: FP4 PL Funding by: NCN US DOE Funding by: ASSL

License: GPL v3

[Tests](#) [CI](#) [PyPI package 2.131](#) [Source](#) [Documentation](#)

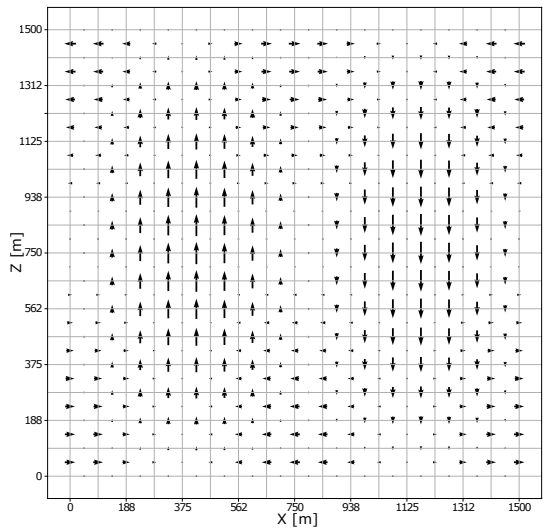
PySDM is a package for simulating the dynamics of population of particles. It is intended to serve as a building block for simulation systems modelling fluid flows involving a dispersed phase, with PySDM being responsible for representation of the dispersed phase. Currently, the development is focused on atmospheric cloud physics applications, in particular on modelling the dynamics of particles immersed in moist air using the particle-based (a.k.a. super-droplet) approach to represent aerosol/cloud/rain microphysics. The package features a Pythonic high-performance implementation of the Super-Droplet Method (SDM) Monte-Carlo algorithm for representing collisional growth (Shima et al. 2009), hence the name.

PySDM documentation is maintained at: <https://open-atmos.github.io/PySDM>

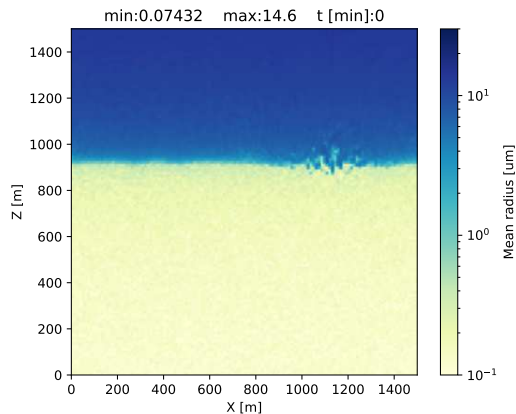
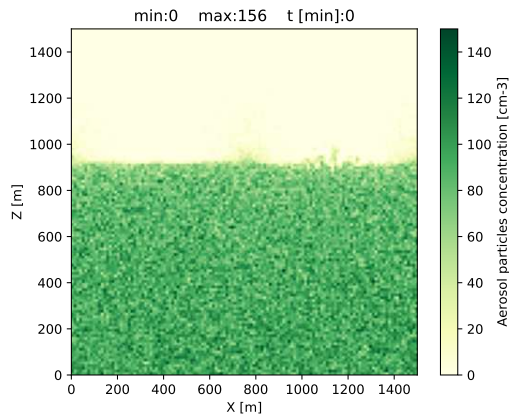
There is a growing set of [example Jupyter notebooks](#) exemplifying how to perform various types of calculations and simulations using PySDM. Most of the example notebooks reproduce results and plot from literature, see below for a list of examples and links to the notebooks (which can be either executed or viewed "in the cloud").

There are also a growing set of [tutorials](#), also in the form of Jupyter notebooks. These tutorials are intended for teaching purposes and include short explanations of cloud microphysical concepts paired with widgets for running interactive simulations using PySDM. Each tutorial also comes with a set of questions at the end that can be used as homework problems. Like the examples, these tutorials can be executed or viewed "in the cloud" making it an especially easy way for students to get started.

Kessler kinematic test (particle-based μ -physics setup as in Arabas et al. '15)



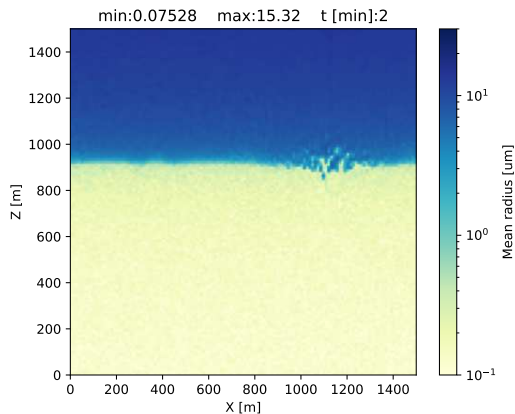
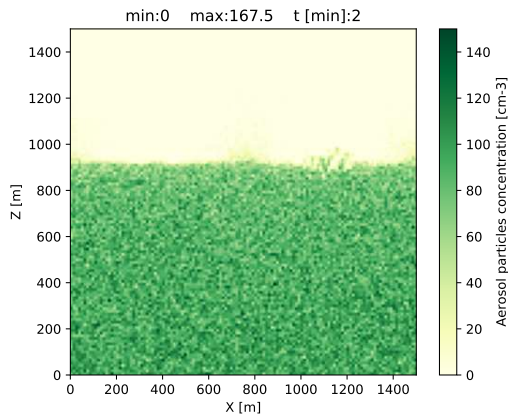
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2²¹

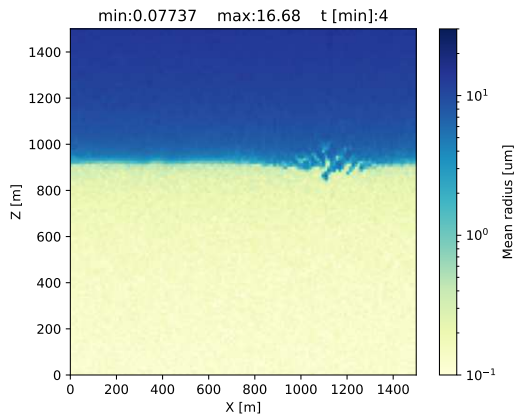
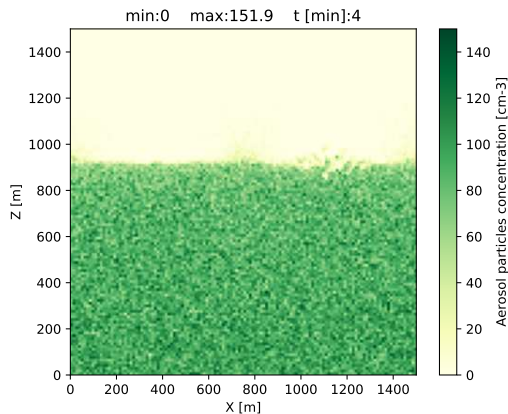
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2^{21}

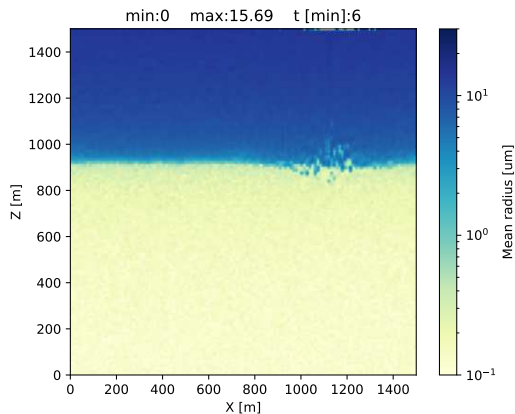
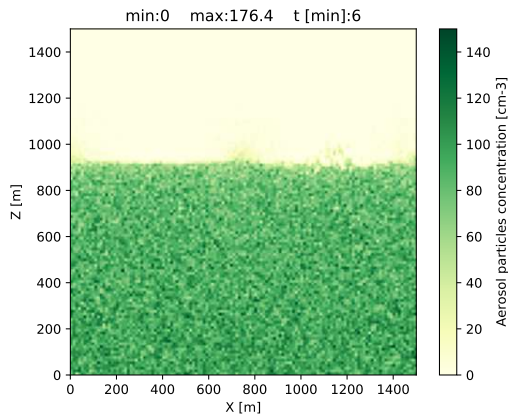
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2²¹

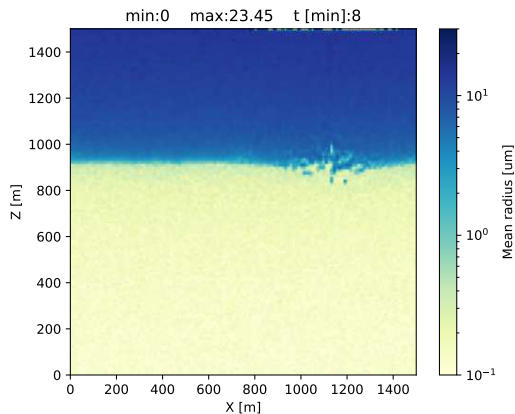
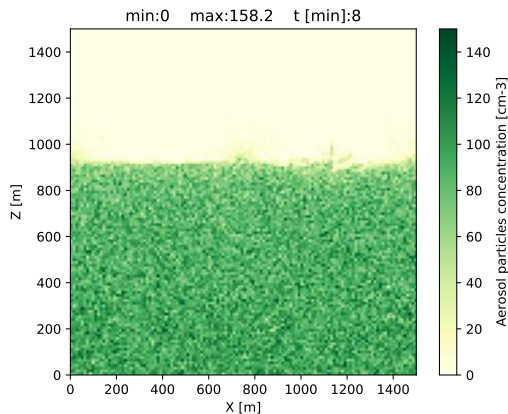
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2^{21}

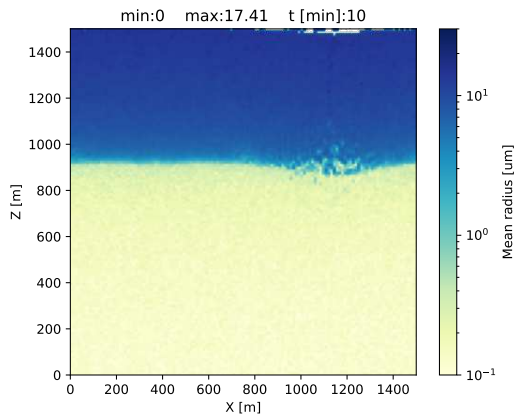
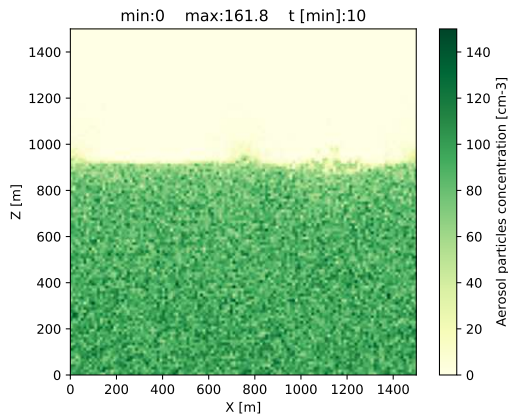
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2²¹

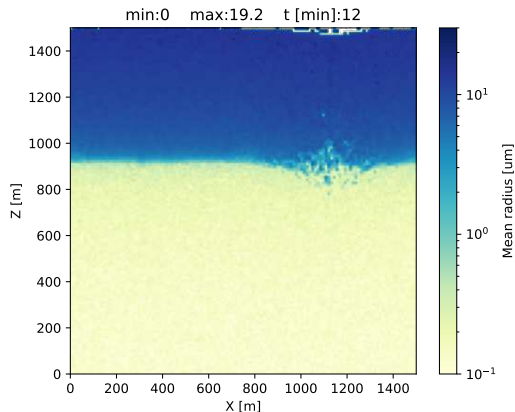
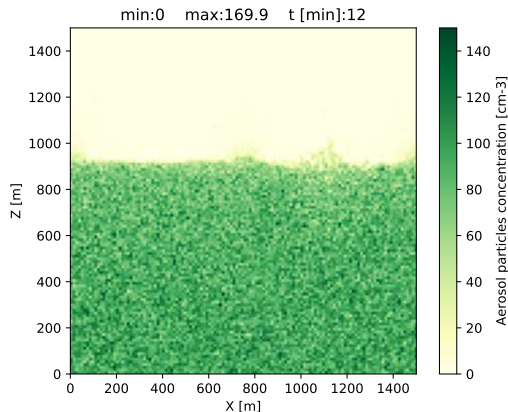
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2^{21}

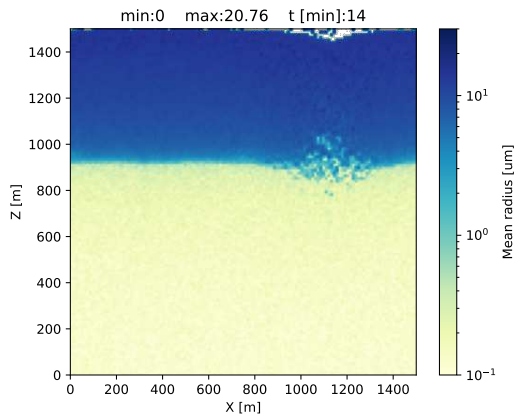
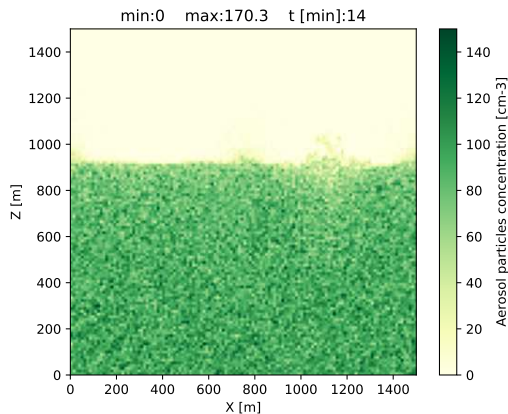
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2²¹

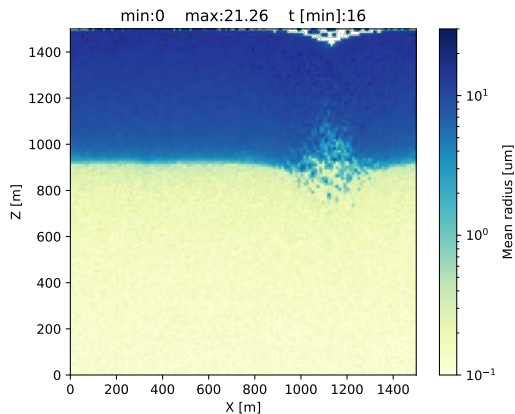
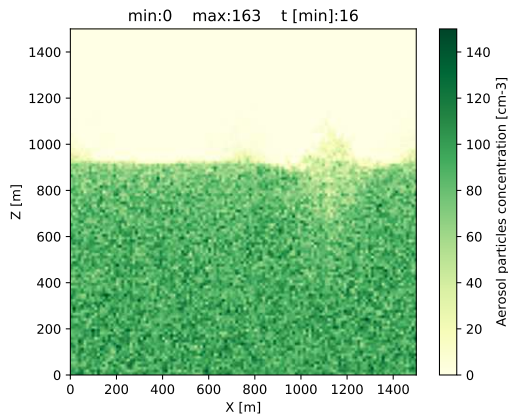
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2²¹

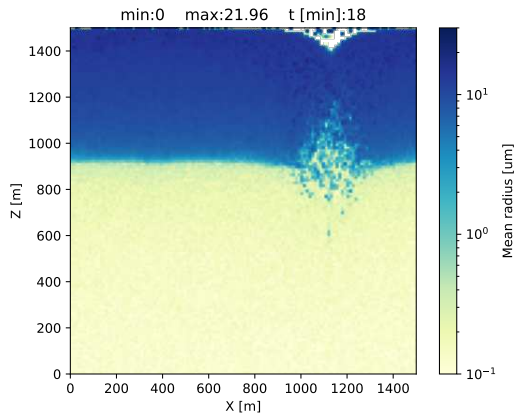
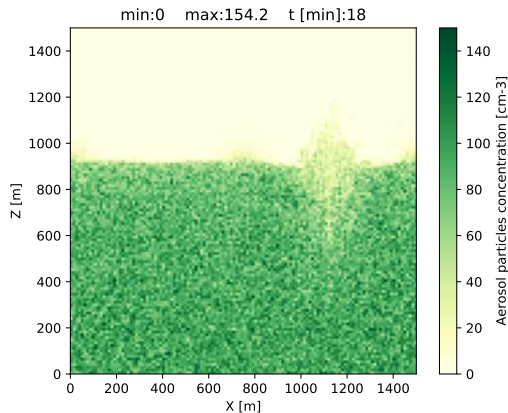
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2^{21}

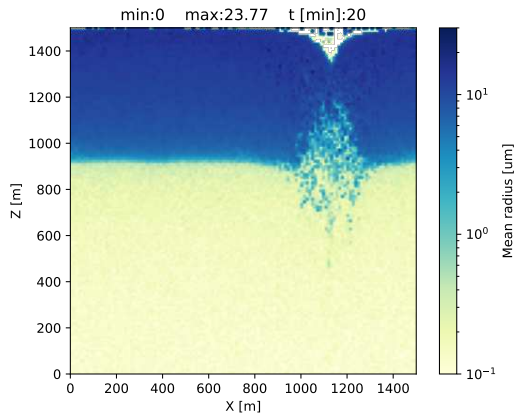
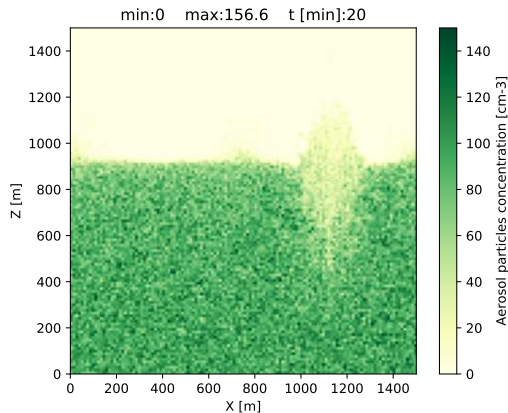
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2^{21}

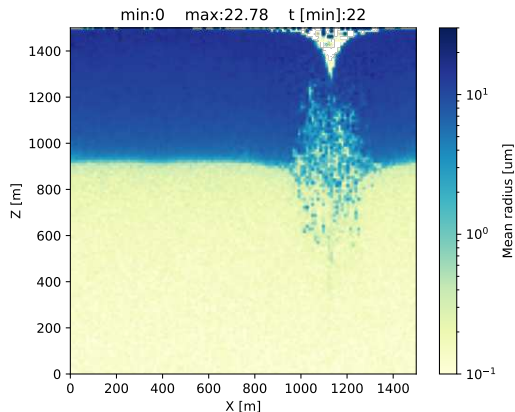
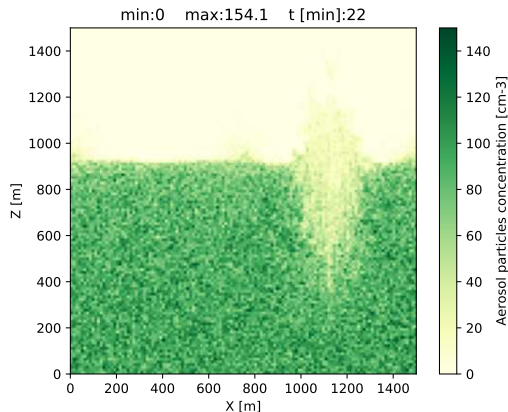
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2²¹

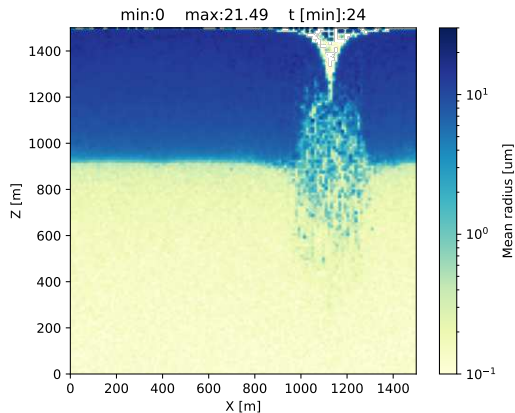
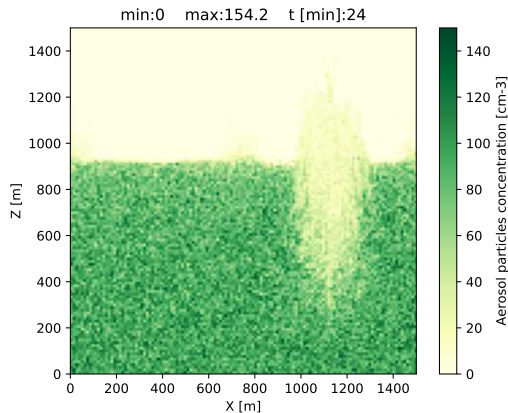
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2²¹

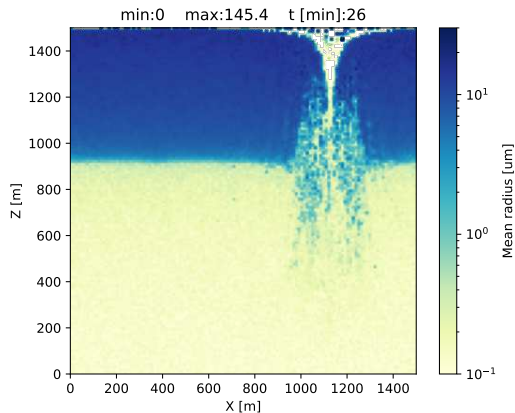
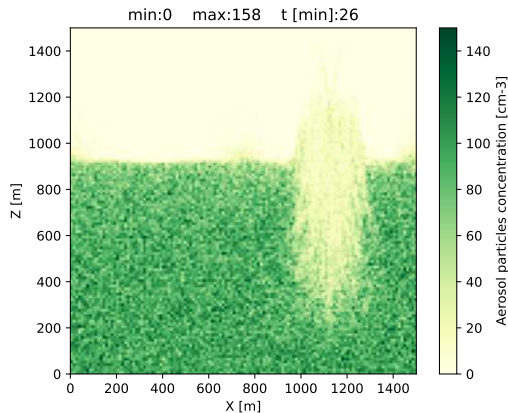
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2^{21}

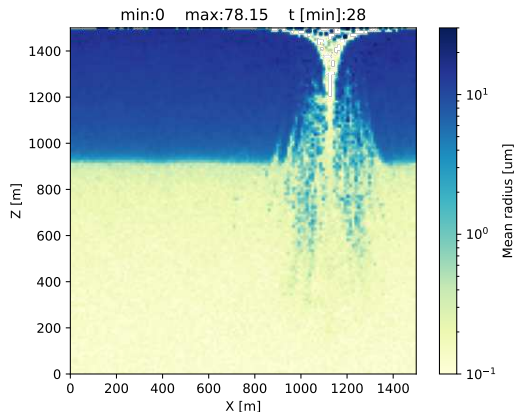
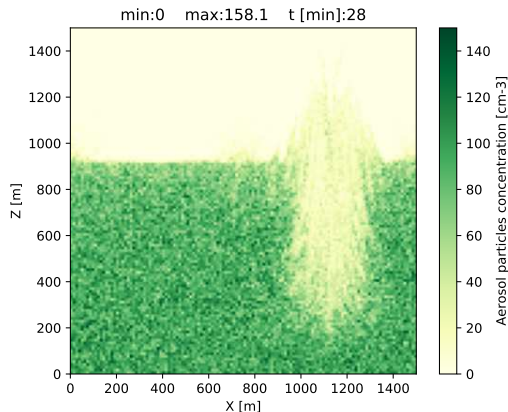
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2²¹

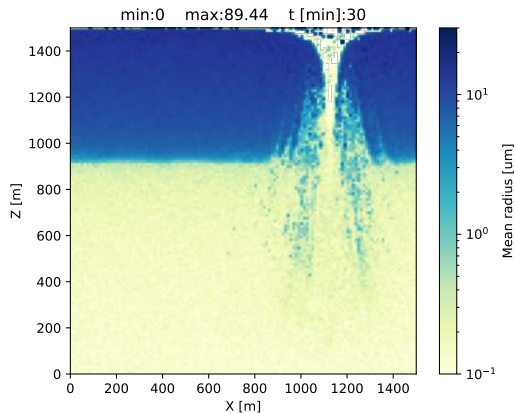
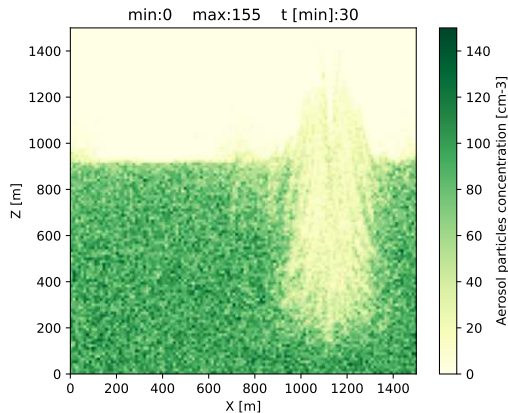
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2^{21}

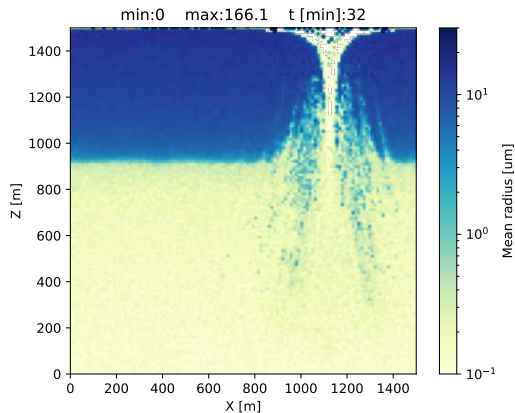
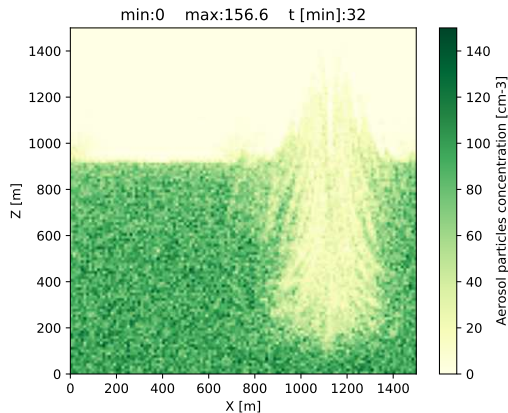
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2^{21}

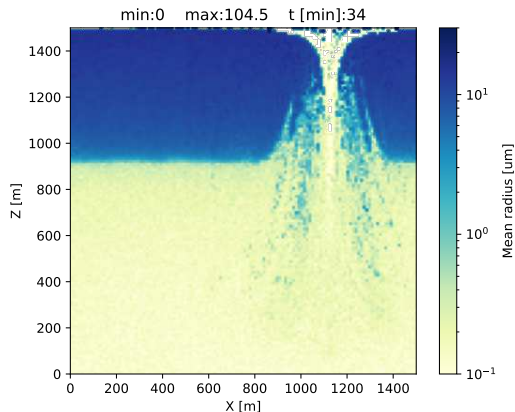
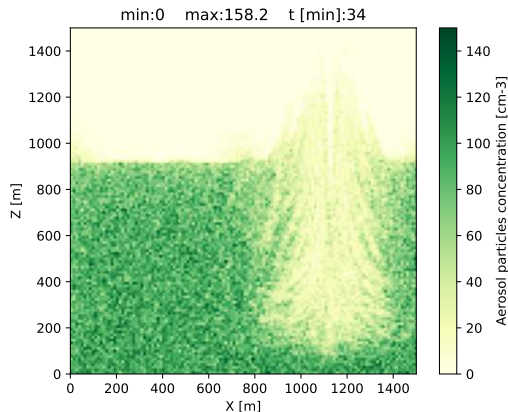
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2²¹

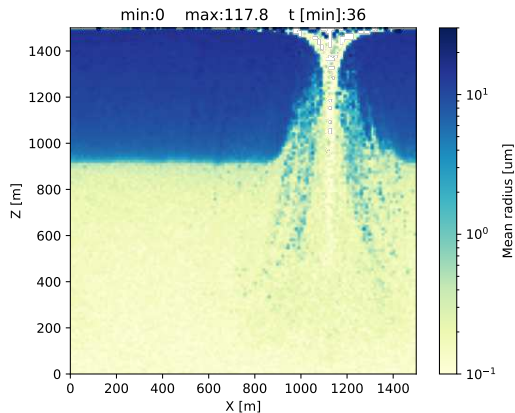
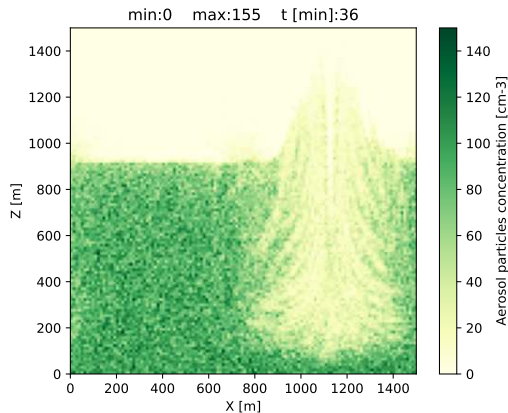
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2^{21}

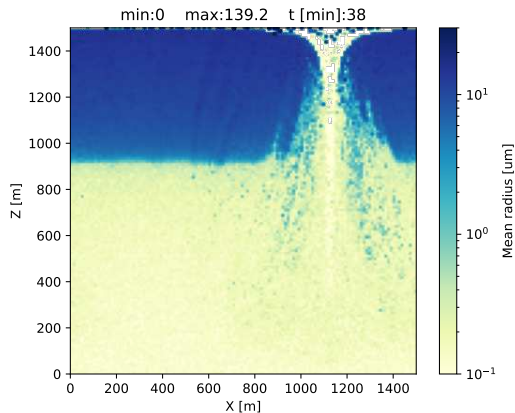
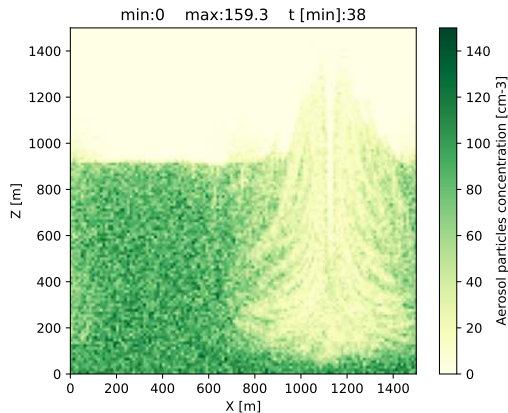
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2^{21}

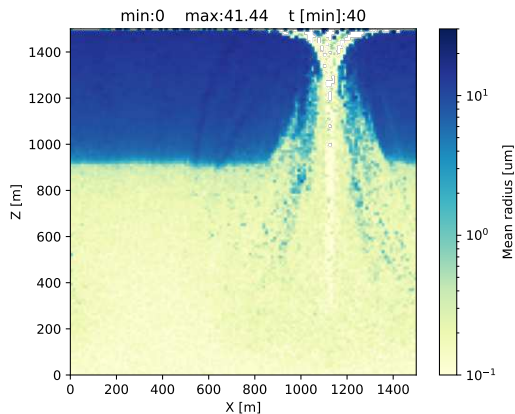
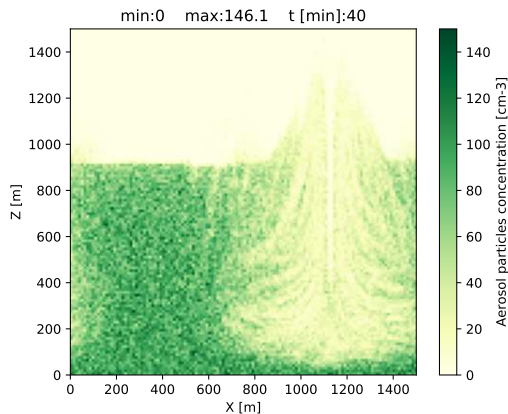
prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2^{21}

prescribed-flow Sc (incl. virga/CCN resuspension) – PySDM + PyMPDATA



Computational grid: 128x128

Computational particles: 2²¹

Geosci. Model Dev., 16, 4193–4211, 2023
<https://doi.org/10.5194/gmd-16-4193-2023>
© Author(s) 2023. This work is distributed under
the Creative Commons Attribution 4.0 License.



Geoscientific
Model Development
Open Access
EGU

Development and technical paper

Breakups are complicated: an efficient representation of collisional breakup in the superdroplet method

Emily de Jong¹, John Ben Mackay^{2,a}, Oleksii Bulenok³, Anna Jaruga⁴, and Sylwester Arabas^{5,b,c}

¹Department of Mechanical and Civil Engineering, California Institute of Technology, Pasadena, CA, USA

²Scripps Institution of Oceanography, San Diego, CA, USA

³Faculty of Mathematics and Computer Science, Jagiellonian University, Kraków, Poland

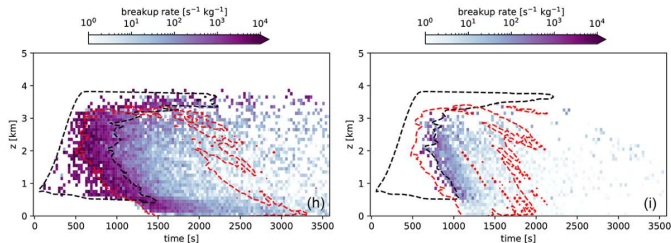
⁴Department of Environmental Science and Engineering, California Institute of Technology, Pasadena, CA, USA

⁵Faculty of Physics and Applied Computer Science, AGH University of Krakow, Kraków, Poland

^aformerly at: Department of Environmental Science and Engineering, California Institute of Technology, Pasadena, CA, USA

^bformerly at: Department of Atmospheric Sciences, University of Illinois Urbana-Champaign, Urbana, IL, USA

^cformerly at: Faculty of Mathematics and Computer Science, Jagiellonian University, Kraków, Poland



- **Numba-MPI**

```
1 import numba
2 from mpi4py.MPI import COMM_WORLD
3
4 def number_crunching():
5     rank = COMM_WORLD.Get_rank()
6
7 numba.njit(number_crunching)()
```

Traceback (most recent call last):

File ".../numba_plus_mpi4py.py", line 7, in <module>

numba.njit(number_crunching)()

File ".../numba/core/dispatcher.py", line 468, in _compile_for_args

error_rewrite(e, 'typing')

File ".../numba/core/dispatcher.py", line 409, in error_rewrite

raise e.with_traceback(None)

numba.core.errors.TypeError: Failed in nopython mode pipeline (step: nopython frontend)

Untyped global name 'COMM_WORLD': Cannot determine Numba type of <class 'mpi4py.MPI.Intracomm'>

File "numba_plus_mpi4py.py", line 5:

def number_crunching():

rank = COMM_WORLD.Get_rank()

^



Original software publication

Numba-MPI v1.0: Enabling MPI communication within Numba/LLVM JIT-compiled Python code

Kacper Derlatka ^{a 1}, Maciej Manna ^{a 2}, Oleksii Bulenok ^{a 3}, David Zwicker ^b, Sylwester Arabas ^c  

^a Faculty of Mathematics and Computer Science, Jagiellonian University in Kraków, Poland

^b Max Planck Institute for Dynamics and Self-Organization, Göttingen, Germany

^c Faculty of Physics and Applied Computer Science, AGH University of Krakow, Poland

<https://doi.org/10.1016/j.softx.2024.101897> 

Under a Creative Commons [license](#) 

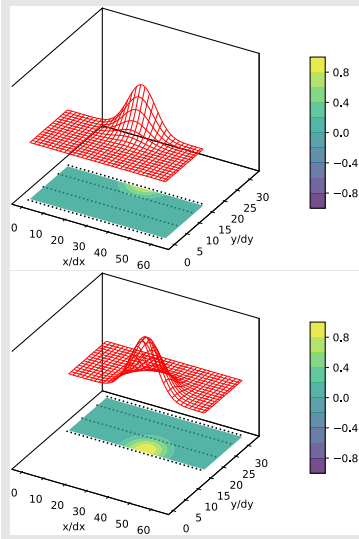
 Open access

Abstract

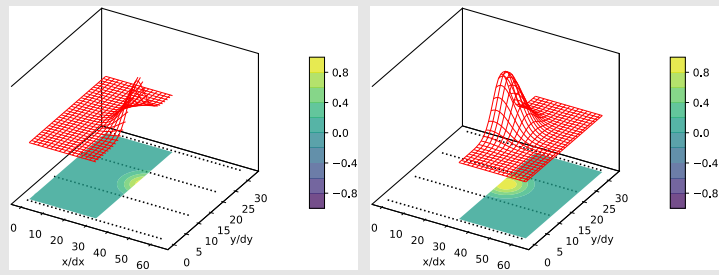
The numba-mpi package offers access to the Message Passing Interface (MPI) routines from Python code that uses the Numba just-in-time (JIT) compiler. As a result, high-performance and multi-threaded Python code may utilize MPI communication facilities without leaving the JIT-compiled code blocks, which is not possible with the mpi4py package, a higher-level Python interface to MPI. For debugging or code-coverage analysis purposes, numba-mpi retains full functionality of the code even if the JIT compilation is disabled.

PyMPDATA-MPI: customisable hybrid threading + MPI parallelisation

threading dim = MPI dim



threading dimension $\neq$ MPI dimension



Derlatka et al. 2024 (SoftwareX, doi:10.1016/j.softx.2024.101897)



[Help](#) [Docs](#) [Sponsors](#) [Log in](#) [Register](#)

numba-mpi 1.3.1

`pip install numba-mpi` 

 Latest release

Released: May 27, 2026

Numba @jittable MPI wrappers tested on Linux, macOS and Windows

Navigation

- Project description**
- Release history
- Download files

Project description



Numba-MPI

Python 3

LLVM

Numba

Linux

macOS

Windows

tests+pypi **failing**

Maintained? **yes**

License: **GPL v3**

pypi package

1.3.1

Anaconda.org

1.3.1

AUR package

DOI

10.5281/zenodo.20410878

Overview

Numba-MPI provides Python wrappers to the C MPI API callable from within [Numba JIT-compiled code](#) (@jit mode). For an outline of the project, rationale, architecture, and features, refer to: [Numba-MPI paper in SoftwareX \(open access\)](#) (please cite if Numba-MPI is used in your research).

Support is provided for a subset of MPI routines covering: `size/rank`, `send/recv`, `sendrecv`, `allreduce`, `reduce`, `bcast`, `scatter/gather` & `allgather`, `barrier`, `wtime` and basic asynchronous communication with `isend/irecv` (only for contiguous arrays); for request handling including `wait/waitall/waitany` and `test/testall/testany`.

Verified details

 These details have been [verified by PyPI](#)

Project links

- Documentation
- Source
- Tracker

GitHub Statistics

- Repository
- Stars: 47

pypi.org/p/PyMPDATA

pypi.org/p/PySDM

pypi.org/p/Numba-MPI

Thanks to all (>20 students) code contributors!

Thank you for your attention!

open-atmos-krk.github.io